

Relatório: Decisões de Implementação da Etapa 2

1. Organização Geral da Solução

A aplicação foi implementada com FastAPI no back-end, PostgreSQL como banco de dados e SQLAlchemy como ferramenta de mapeamento objeto-relacional. A estrutura separa as responsabilidades entre modelos, schemas, rotas, migrações e testes.

As entidades do domínio hospitalar foram representadas como classes Python em `app/models/models.py`. Classes como `Paciente`, `Atendimento`, `Internacao`, `Procedimento` e `Escala` correspondem às tabelas do banco. Seus atributos representam colunas, enquanto `mapped_column`, chaves primárias, chaves estrangeiras e `relationship()` representam as restrições e relacionamentos do modelo relacional.

Também foi utilizada herança entre entidades. `Paciente` e `Profissional` herdam de `Pessoa`, enquanto `Preceptor` e `Residente` herdam de `Profissional`. Essa escolha evita a repetição dos dados comuns, como nome, CPF, telefone e data de nascimento, mantendo a estrutura compatível com o modelo relacional.

A aplicação utiliza sessões do SQLAlchemy por requisição. A dependência `get_db()` cria uma sessão, disponibiliza-a para a rota e garante seu fechamento ao final. As operações são confirmadas com `commit()` e, em caso de erro, desfeitas com `rollback()`. Em operações que precisam gerar identificadores antes da conclusão da transação, como o cadastro de paciente e suas alergias, é utilizado `flush()`.

2. Escolha do SQLAlchemy (ORM) e Estruturas Analíticas

A escolha do SQLAlchemy foi motivada pela integração com Python e FastAPI, pelo suporte a tipagem, pela representação das entidades como classes e pelo controle das consultas e transações. A ORM reduz a necessidade de escrever SQL repetitivo para operações CRUD, mantendo as regras de relacionamento próximas ao código da aplicação.

Outro benefício é a possibilidade de combinar abstração e controle. Consultas simples podem ser feitas por meio de `db.query()` ou `select()`, enquanto consultas mais específicas utilizam `JOIN`, funções de agregação e SQL parametrizado. Foi utilizando essa expressividade da ORM que implementamos as consultas avançadas do sistema em Python puro, como a listagem de preceptores de pacientes flamenguistas, o detalhamento do último atendimento por paciente e o cálculo do percentual de procedimentos de alto risco realizados por residente.

A solução não depende exclusivamente da ORM. Recursos específicos do PostgreSQL, como `JSONB`, views, funções PL/pgSQL e stored procedures, são definidos nas migrações do Alembic e acessados pela aplicação por meio de `db.execute(text(...))`. Essa abordagem híbrida foi escolhida porque algumas operações são mais adequadas ao banco de dados do que ao código Python.

As relações ORM foram configuradas com o comportamento padrão de lazy loading. Assim, os relacionamentos só são carregados quando acessados. Para consultas que exibem informações relacionadas, foram utilizados `JOINS` explícitos. Essa decisão evita carregar automaticamente dados que não serão utilizados, embora consultas futuras com grandes volumes possam exigir `selectinload()` ou `joinedload()` para evitar o problema N+1.

3. Racional de Arquitetura: Triggers vs. Stored Procedures

A decisão principal foi separar as responsabilidades automáticas das operações que precisam ser chamadas explicitamente.

Os **triggers** foram utilizados para regras que devem ser executadas automaticamente sempre que determinados eventos ocorrem no banco. Foram implementados três casos principais:

1. O trigger de sobreposição de escala impede que um residente seja escalado no mesmo dia e turno em unidades diferentes. Essa regra foi colocada no banco porque representa uma restrição de consistência que não deve depender apenas da API. O bloqueio da linha do residente com `FOR UPDATE` também protege a verificação contra alterações concorrentes.
2. O trigger de auditoria registra operações de `INSERT`, `UPDATE` e `DELETE` na tabela `auditoria_atendimento`. A auditoria precisa ocorrer independentemente do sistema que realizou a alteração, seja a API, uma ferramenta administrativa ou outra aplicação. Por isso, o trigger é mais adequado do que implementar o registro somente nas rotas.
3. O trigger de atualização da média de tempo dos procedimentos recalcula `media_tempo_procedimento` após a inserção de um procedimento realizado. A coluna funciona como um dado derivado, facilitando consultas e relatórios posteriores. Como contrapartida, essa estratégia exige que novas regras sejam adicionadas caso também sejam permitidas alterações ou exclusões que afetem essa média.

Os **stored procedures**, por outro lado, foram utilizadas para operações de negócio que devem ser iniciadas explicitamente pela aplicação:

- `sp_registrar_atendimento_completo` cria um atendimento e seus procedimentos em uma única operação;
- `sp_reajustar_escala` altera o horário de uma escala, ignorando conflitos conforme a regra definida;
- `sp_calcular_tempo_medio_espera` calcula uma informação agregada por unidade.

A procedure de atendimento completo é importante porque envolve várias inserções relacionadas. Centralizar essa operação no banco reduz a quantidade de chamadas entre a aplicação e o PostgreSQL e garante que o processo seja executado de forma atômica. Se ocorrer uma falha em qualquer etapa, a transação pode ser revertida.

A função `sp_calcular_tempo_medio_espera`, apesar de estar no conjunto de rotinas armazenadas, é tecnicamente uma função que retorna uma tabela. Ela foi escolhida por representar uma consulta calculada, sem necessidade de alterar dados.

Portanto, triggers e procedures não foram tratados como alternativas excludentes. Os triggers foram usados para regras automáticas, auditoria e manutenção de consistência; as procedures foram usadas para fluxos de negócio e operações analíticas chamadas pela API.

4. Concorrência Multithread, Migrações e Conclusão

Para garantir a estabilidade das transações do sistema sob concorrência de acessos, foi implementado o teste simulado de duas transações concorrentes rodando em threads paralelas, tentando escalar um mesmo residente para o mesmo dia e turno. O uso da trava pessimista com `FOR UPDATE` dentro do trigger de escala evitou condições de corrida, garantindo a gravação da primeira transação e rejeitando a segunda com erro de violação de consistência.

Triggers, procedures, views, tabelas e alterações de colunas são criados por migrações Alembic. Isso permite reproduzir a estrutura do banco em diferentes ambientes, controlar a ordem de criação dos objetos e realizar o downgrade quando necessário.

A solução adota, portanto, uma arquitetura híbrida: a ORM facilita o desenvolvimento da aplicação e o acesso às entidades, enquanto o PostgreSQL concentra regras que exigem consistência, atomicidade, auditoria ou melhor desempenho. Essa divisão aumenta a confiabilidade do sistema, reduz duplicação de regras e aproveita os recursos específicos do banco de dados sem abandonar a organização e a produtividade oferecidas pelo SQLAlchemy.